

Trabajo Fin de Máster: hipótesis, técnica y resultados

Este documento sigue los tres apartados que piden las instrucciones del TFM: la hipótesis que se quiere contrastar, la técnica elegida y por qué es adecuada, los resultados y las conclusiones. Todo el análisis se puede reproducir en el cuaderno [notebooks/tfm_graficos.ipynb](#), donde están los gráficos, las tablas y los contrastes estadísticos que se citan aquí. El cuaderno también puede abrirse directamente en Google Colab desde [este enlace](#), y el resto del código del estudio está en el mismo repositorio, [github.com/cecimerelo/ml-sandbox](#).

La aplicación, con el formulario, está desplegada en [ml-sandbox-7cn3.onrender.com](#). Está en el plan gratuito de Render, que apaga el servicio cuando nadie lo usa, así que la primera carga puede tardar alrededor de un minuto.

1. Objetivo e hipótesis

Elegir qué método de aprendizaje supervisado usar para un problema concreto no es sencillo, sobre todo para quien empieza. Hay decenas de métodos, cada uno con sus supuestos, y la respuesta depende del tamaño de los datos, del tipo de variables o de si hace falta poder explicar el modelo. Este trabajo aborda esa elección con dos piezas. La primera es un estudio benchmark, es decir, entrenar y evaluar muchos métodos en muchos datasets para saber cuál funciona mejor en cada uno. La segunda es una aplicación web en la que el usuario describe su problema mediante un formulario, o sube directamente sus datos, y recibe tres métodos recomendados, con una explicación de por qué, que después puede entrenar sobre sus propios datos.

La recomendación de la aplicación se organiza en dos capas. La primera, que llamo Layer 1, aplica las reglas de selección de método que enseña ISLR (*An Introduction to Statistical Learning*), el libro de referencia del curso, junto con las restricciones que indique el usuario, como la de necesitar un modelo interpretable. La segunda, Layer 2, es el objeto de este trabajo: un modelo entrenado con los resultados del benchmark que, a partir de las características del problema, como el tamaño de la muestra, el número de predictores, el tipo de variable respuesta, la presencia de valores faltantes o el balance de clases, predice qué métodos van a funcionar mejor.

La **hipótesis principal** es que esta recomendación personalizada acierta el mejor método con más frecuencia que una recomendación fija, es decir, que recomendar siempre el mismo método sea cual sea el problema. Si no fuera así, estudiar el problema del usuario no aportaría nada y bastaría con recomendar siempre lo mismo. Como hipótesis secundaria planteo que las heurísticas de ISLR, las reglas de la primera capa, aciertan más que una elección al azar. Son las reglas que se enseñan en el curso, y el benchmark permite comprobar si se sostienen frente al rendimiento real de los métodos.

Las dos hipótesis se pueden contrastar de forma empírica. Basta con tomar una colección de datasets lo bastante amplia y variada y comparar, para cada estrategia de selección, con qué frecuencia acierta el método que de verdad es el mejor en cada dataset y cuánto rendimiento se pierde cuando no lo hace.

2. Técnica utilizada y por qué es apropiada

Origen de los datos

Los datos vienen de tres colecciones públicas muy usadas en la literatura. [OpenML-CC18](#) es de clasificación y [OpenML-CTR23](#) de regresión, las dos con al menos 500 filas por dataset. [PMLB](#) aporta datasets de ambos tipos con menos de 500 filas. La idea era reunir datasets que cubrieran todos los casos que un usuario puede describir en el formulario de la aplicación.

Un dataset es elegible si cumple todos estos criterios:

- Tiene entre 50 y 100.000 filas. Por debajo de 50, cada partición de la validación cruzada de 5 particiones tendría menos de 10 filas y la puntuación dependería sobre todo de qué filas cayeran en cada una. Por encima de 100.000, un solo dataset consumiría más cómputo que toda la banda pequeña.
- Tiene como mucho 500 predictores, el mismo límite que acepta la aplicación.
- No está construido a partir de imágenes (MNIST, Fashion-MNIST, Devnagari-Script, CIFAR-10), porque cada columna es un píxel y el estudio trata de datos tabulares.
- Contiene datos medidos, no generados. Por eso quedan fuera los datasets sintéticos de PMLB: las ecuaciones de física de Feynman y Strogatz y los datasets BNG, muestreados de redes bayesianas.
- Sus propios autores no lo han retirado. PMLB marca como obsoletos algunos datasets y esos no entran.
- Tiene completos los metadatos, incluido el número de filas.
- Si viene de PMLB, tiene menos de 500 filas, porque esa colección solo se usa para cubrir la banda que no cubren OpenML-CC18 ni OpenML-CTR23.

Además, de cada familia de datasets (varias versiones de la misma fuente, como los `analcat-data_*` o los `mfeat-*`) entran como mucho dos, para que una banda no parezca variada cuando en realidad repite el mismo tipo de datos.

Estos criterios no tienen todos el mismo origen, y conviene distinguirlos. El límite de 500 predictores y la exclusión de imágenes no se fijaron para el estudio: son el alcance que el documento de requisitos de la aplicación ya declaraba antes de empezarlo, así que la selección no puede haberse ajustado para favorecer a ningún método. El umbral de 500 filas es también la banda más baja del formulario de la aplicación, y coincide con el criterio de los propios autores de OpenML-CC18, que descartan como demasiado pequeño todo dataset por debajo de esa cifra ([generador de la colección](#)). Por eso hace falta PMLB para cubrir esa banda. El resto son decisiones propias del estudio: los límites de 50 y 100.000 filas, y la exclusión de datasets sintéticos, retirados o de una misma familia. Todas están justificadas en el registro

de decisiones del repositorio ([DECISIONS.md](#), entradas D-004, D-016, D-025 y D-054). La de las familias es la más débil de todas: la familia se deduce del nombre del dataset, y el límite de dos no responde a ningún criterio externo.

De 267 candidatos, 195 cumplían los criterios de elegibilidad y me quedé con 106, estratificados por tamaño y por tipo de tarea con el objetivo de tener 20 datasets en cada combinación. En las bandas de menos de 500 filas y de 500 a 10.000 filas el objetivo se cumplió exactamente, con 40 datasets en cada una. En la de más de 10.000 filas no había suficientes candidatos, así que entraron todos los que cumplían los criterios, que fueron 26. La colección final tiene 55 datasets de regresión y 51 de clasificación, con tamaños que van de 50 a 96.320 filas y de 1 a 216 predictores. Para que el cómputo fuera asumible, cada dataset se evalúa como mucho sobre 20.000 filas, elegidas al azar con la semilla del estudio. Como el límite es el mismo para todos los métodos, no cambia la comparación entre ellos.

La colección no se reunió de una vez. La primera versión del estudio tenía 60 datasets, y las primeras evaluaciones se hicieron incluso antes de completarla, con 44. Después se amplió a 106, ampliando por igual todos los grupos de tamaño y de tarea, para comprobar si los resultados de esa primera versión se mantenían con una muestra casi el doble de grande. Varios de ellos no se mantuvieron, como se explica más adelante.

Hay que tener en cuenta que el equilibrio solo se buscó en esas dos variables. Algunos regímenes tienen pocos representantes: solo 6 datasets superan los 50 predictores, 13 tienen pocas observaciones por predictor y apenas 8 traen valores faltantes de origen. Para estudiar este último caso repetí el benchmark eliminando al azar el 5 % y el 25 % de las celdas de los predictores.

Benchmark

En cada dataset entrené y evalué 20 métodos del temario de ISLR: regresión lineal, Ridge, Lasso, regresión logística, LDA, QDA, Naive Bayes, KNN, regresión polinómica con y sin interacciones, splines, PCR, PLS, árbol de decisión, bagging, random forest, gradient boosting, SVM lineal, SVM con núcleo RBF y una red neuronal. En total son 38.820 evaluaciones.

La métrica decide cuál es el mejor método en cada dataset, y todo el estudio se apoya en esa respuesta, así que la elegí con dos condiciones: que no diera por bueno un modelo trivial y que permitiera promediar resultados entre datasets distintos.

En clasificación usé la *balanced accuracy*, que es la media del *recall* de cada clase. La colección tiene datasets con clases muy desequilibradas, y en ellos la *accuracy* normal pondría en cabeza a un modelo que predice siempre la clase mayoritaria, justo donde más importa, el mejor método sería el equivocado. La *balanced accuracy* no tiene ese problema, sirve igual para problemas binarios y multiclase, va de 0 a 1 y, cuando las clases están equilibradas, coincide con la *accuracy*, así que no se pierde nada en los casos fáciles. Descarté el ROC-AUC porque necesita probabilidades, y obtenerlas de la SVM obliga a una validación cruzada interna que multiplica el coste del método más caro. En multiclase, además, hay que elegir

cómo promediarlo. El F1-macro y el coeficiente de correlación de Matthews eran defendibles, pero más difíciles de explicar sin aportar nada en este caso.

En regresión usé el R^2 . Para decidir qué método gana dentro de un dataset la escala da igual, pero el *regret*, que se define más abajo, promedia diferencias de rendimiento entre datasets. Con el RMSE o el MAE eso supondría sumar errores medidos en unidades distintas, por ejemplo precios de viviendas con concentraciones químicas, y el resultado no significaría nada. El R^2 no depende de la escala y es además la métrica que usa ISLR. Un modelo muy malo puede tener un R^2 muy negativo y arrastrar cualquier media, así que lo acoté en 0: un método que lo hace peor que predecir siempre la media cuenta como si predijera la media. Solo ajusté por validación cruzada interna los hiperparámetros sin los que un método no está definido, como la penalización de Ridge o de Lasso. El resto de métodos usa los valores por defecto de la librería, que es también lo que hace ISLR.

En los 66 datasets de OpenML usé las 10 particiones de validación cruzada que publica la propia plataforma, para que los resultados se puedan comparar con otros trabajos sobre esas colecciones. Los 40 datasets de PMLB no traen particiones, así que generé 5, estratificadas en clasificación y con semilla fija. Elegí 5 porque era el valor que venía por defecto en la configuración del estudio, y además es uno de los dos que recomienda ISLR. Mezclar los dos esquemas no introduce sesgo, porque todas las métricas se calculan dentro de cada dataset antes de juntarlas.

Dos métodos se consideran empatados cuando sus puntuaciones medias se separan menos de una desviación estándar, tomando la del mejor método. Por eso un mismo dataset puede tener varios mejores métodos. Con todo esto se obtiene un ranking real de los métodos en cada dataset, que es la referencia contra la que se juzga cualquier estrategia de selección.

Estrategias comparadas

Sobre ese ranking comparo cuatro formas de elegir método. La primera es la estrategia aprendida, Layer 2, que es la recomendación personalizada y el objeto del trabajo. Es un *random forest* que predice cuánto rendimiento perderá cada método frente al mejor y recomienda los que menos pierden. Lo hace a partir de siete características del dataset: el tipo de tarea (regresión, clasificación binaria o multiclase), el número de filas, el número de predictores, la relación entre observaciones y predictores, el tipo de variables (numéricas, categóricas o mixtas), la cantidad de valores faltantes y el balance de clases. Son exactamente las que el usuario puede indicar en el formulario, y el modelo las recibe agrupadas en los mismos intervalos que ofrece el formulario, por ejemplo menos de 500 filas, de 500 a 10.000 o más de 10.000. Así la recomendación funciona igual aunque el usuario no suba sus datos. Elegí un random forest porque esa pérdida depende de combinaciones de características. Tener pocas filas, por ejemplo, no perjudica igual a una red neuronal que a un árbol de decisión, y un bosque de árboles aprende ese tipo de relaciones por sí solo, cosa que un modelo lineal no hace.

La segunda estrategia recomienda siempre el mismo método, el que menos rendimiento pierde de media en los datasets de entrenamiento, que en este estudio resulta ser Random Forest. Es la comparación que de verdad decide la hipótesis, porque si la recomendación personali-

zada no la supera, analizar el problema del usuario no aporta nada. La tercera son las heurísticas de ISLR, es decir, la primera capa de la aplicación. Son reglas fijas extraídas del libro que puntúan cada método según las características del problema. Por ejemplo, con pocas observaciones favorecen los métodos sencillos o regularizados, porque los flexibles acaban ajustando el ruido. Cuando el número de predictores se acerca al de observaciones, favorecen la regularización, como Ridge o Lasso. Y con muchas observaciones por predictor favorecen los métodos flexibles, que ya tienen datos suficientes. Estas reglas no aprenden nada del benchmark. La cuarta estrategia elige al azar y sirve como límite inferior.

Todas las estrategias se evalúan dejando fuera un dataset cada vez. Para cada uno de los 106 datasets, la estrategia se construye con los otros 105 y se le pide una recomendación para el que ha quedado fuera. Así se reproduce la situación real, en la que el recomendador tiene que acertar con un problema que nunca ha visto. Lo apliqué a todas las estrategias, también a las que no aprenden nada, para que ninguna partiera con ventaja. Gracias a eso encontré y corregí un error por el que la estrategia fija elegía su método mirando también el dataset con el que luego se la evaluaba.

Uso tres medidas. La tasa de acierto indica en qué proporción de datasets la primera recomendación está entre los mejores métodos. El acierto en las tres primeras indica si algún mejor método aparece entre las tres sugerencias, que es lo que enseña la aplicación. El regret mide cuánto rendimiento se pierde por seguir la primera recomendación en lugar de usar el mejor método. Está en la misma escala que la métrica, de 0 a 1, así que un regret de 0,03 significa perder tres centésimas de *balanced accuracy* o de R^2 . Si la recomendación es un método que no llega a ejecutarse en ese dataset, se considera que se pierde la puntuación entera.

Todos los resultados se dan dos veces: sobre los 106 datasets y sobre los 51 en los que la elección importa de verdad. Dos ejemplos de la colección ayudan a ver la diferencia. En `forest_fires`, que consiste en predecir el área quemada en incendios forestales, ningún método consigue predecir nada: el mejor obtiene un R^2 de 0,019 y los peores 0,000, así que la regla de empate cuenta a los 16 métodos como mejores y cualquier estrategia acierta, incluso eligiendo a ciegas. En `611_fri_c3_100_5`, en cambio, la red neuronal obtiene 0,892 y el segundo mejor método, KNN, 0,758, de modo que solo acierta quien elige la red neuronal. Si se promedian los dos tipos de dataset, los del primero regalan un acierto a todas las estrategias por igual y las diferencias entre ellas se diluyen, igual que en un examen en el que la mitad de las preguntas diera por buena cualquier respuesta. La elección al azar lo deja claro: acierta en el 49 % de los datasets del primer tipo y solo en el 2 % de los del segundo.

Por eso separé los datasets en los que empatan como mucho tres métodos, que son 51, de los otros 55. El umbral de tres no se eligió para mejorar los resultados: es el número de sugerencias que muestra la aplicación, y si empatan más de tres métodos, un usuario que siga cualquiera de ellas acierta seguro, así que ese dataset no pone a prueba a ninguna estrategia. La separación afecta solo a la evaluación, porque todas las estrategias se entrenan siempre con todos los datasets, y se informan siempre los dos grupos porque responden a preguntas distintas: los 106 dicen con qué frecuencia importa elegir bien, y los 51 si la estrategia acierta cuando importa.

Esta separación no estaba en el planteamiento inicial. Surgió al revisar la primera evaluación, hecha con 44 datasets, en la que la elección al azar acertaba en las tres primeras sugerencias el 75 % de las veces, algo imposible si de verdad hubiera que acertar un único método entre quince. Al investigarlo vi que la causa eran los empates, y a partir de ahí se decidió informar siempre de los dos grupos. Aunque la decisión se tomó después de ver esos primeros resultados, no los favoreció: el umbral quedó fijado por la interfaz y no se volvió a tocar, y la ventaja que la separación parecía dar entonces a la estrategia aprendida desapareció después al ampliar la colección a 106 datasets. El razonamiento completo está en la decisión D-036 de [DECISIONS.md](#).

Para decidir si las diferencias son reales uso tests pareados, que comparan a dos estrategias sobre los mismos datasets. Para los aciertos uso el test exacto de McNemar, que solo tiene en cuenta los datasets en los que una estrategia acierta y la otra no, y para el regret el test de Wilcoxon de rangos con signo, que compara lo que pierde cada una dataset a dataset. Considero significativa una diferencia cuando p es menor que 0,05. Además calculo intervalos de confianza al 95 % por bootstrap, remuestreando los datasets 2.000 veces, para ver cuánto podrían variar las cifras con otra muestra.

Esta técnica es la adecuada porque la hipótesis no pregunta si un método concreto funciona bien en un dataset concreto, algo que se resolvería con una validación cruzada normal. Pregunta si una forma de elegir método es mejor que otra más simple en general, y eso obliga a repetir la validación en muchos datasets, a tratar a todas las estrategias exactamente igual y a usar un contraste estadístico que separe una diferencia real del ruido de la muestra.

3. Resultados y conclusiones

La tabla recoge los resultados de cada estrategia en los dos grupos de datasets: los 106 de la colección y los 51 en los que la elección de método importa.

Estrategia	Acierto (106)	Acierto (51)	Tres primeras (106)	Tres primeras (51)	Regret (106)	Regret (51)
Aprendida (Layer 2)	0,63	0,49	0,78	0,69	0,041	0,038
Método único fijo	0,62	0,47	0,75	0,65	0,036	0,031
Heurísticas (ISLR)	0,32	0,18	0,68	0,49	0,116	0,112
Aleatoria	0,26	0,02	0,59	0,29	0,244	0,325

La tabla muestra por qué conviene separar los dos grupos. Sobre los 106 datasets el azar parece razonable, con un 0,26 de acierto y un 0,59 en las tres primeras sugerencias, pero en los 51 en los que la elección importa cae a 0,02 y 0,29. Casi todos sus aciertos venían de datasets en los que cualquier método valía. Al pasar de un grupo a otro bajan todas las estrategias, pero la aprendida y la fija siguen empatadas en los dos.

Las cifras de los 51 datasets tienen bastante incertidumbre, porque salen de una muestra pequeña. Los intervalos de confianza al 95 %, calculados por bootstrap sobre los datasets, lo muestran bien: el 0,49 de acierto de la estrategia aprendida podría estar en realidad entre 0,35 y 0,63, y el 0,47 de la fija entre 0,33 y 0,61. Los dos intervalos prácticamente coinciden, así que la diferencia entre ambas no es distinguible del azar.

La hipótesis principal no se cumple. Al comparar la estrategia aprendida con la fija dataset a dataset, ninguna diferencia resulta significativa en ninguno de los dos grupos. En los 51 datasets en los que la elección importa, los casos en que una acierta y la otra no están casi igualados, 4 frente a 3 ($p = 1,000$). En las tres primeras sugerencias la aprendida va algo por delante, 3 frente a 1, pero tampoco es significativo ($p = 0,625$), y lo mismo pasa con el regret ($p = 0,546$). En los 106 el panorama es el mismo: 7 frente a 6 en la tasa de acierto ($p = 1,000$), 5 frente a 1 en las tres primeras ($p = 0,219$) y ninguna diferencia significativa en el regret ($p = 0,485$). La estrategia aprendida comete además algunos errores más graves que la fija, con un peor caso de 0,50 frente a 0,28 sobre los 106 datasets y de 0,39 frente a 0,19 sobre los 51, y eso explica que su regret medio sea un poco más alto aunque acierte con la misma frecuencia.

El resultado no cambia al separar los datasets por tamaño. En la banda de 500 a 10.000 filas, una versión anterior del estudio con 60 datasets parecía mostrar una ventaja de la personalización, pero desapareció al ampliar la colección a 106 y ahora el recuento es de 2 a 2 ($p = 1,000$). Tampoco cambia con el 5 % ni con el 25 % de celdas faltantes, donde ningún contraste sale significativo. La conclusión es que estudiar las características del problema y elegir el método en función de ellas no mejora, ni en aciertos ni en rendimiento perdido, a recomendar siempre el mismo método.

La hipótesis secundaria se cumple solo en parte. Frente al azar, las heurísticas de ISLR pierden bastante menos rendimiento, y la diferencia es significativa tanto en los 106 datasets ($p = 0,002$) como en los 51 en los que la elección importa ($p = 0,001$). En estos últimos también aciertan el mejor método más a menudo, 9 veces frente a 1 ($p = 0,02$). En el conjunto de los 106, en cambio, la diferencia en aciertos no es significativa ($p = 0,41$), porque ahí el azar acierta muchas veces gracias a los empates. Frente a la estrategia fija, las heurísticas quedan claramente por detrás tanto en aciertos como en rendimiento perdido ($p < 0,001$). Es decir, contienen información útil sobre qué método conviene, pero como forma de elegir método están muy lejos de la alternativa más sencilla.

El resultado más interesante del estudio no estaba previsto. Además de poner a prueba las dos hipótesis, el benchmark descubrió cuál es la elección fija que empatiza con la recomendación personalizada y supera a las heurísticas: Random Forest, que está entre los mejores métodos en 66 de los 106 datasets, seguido muy de cerca por Gradient Boosting, con 62. Visto así, un resultado que parece negativo aporta algo positivo, porque mide cuánto importa de verdad elegir bien el método en datos tabulares habituales, y resulta ser menos de lo que se suele pensar.

También medí el coste de exigir un modelo interpretable. En la aplicación, el usuario puede indicar que la interpretabilidad es algo importante, y entonces se descartan los métodos opacos, cuyo razonamiento no puede leerse: bagging, random forest, gradient boosting, SVM con

núcleo RBF y la red neuronal. Si indica que es crítica, se descartan también los que solo se explican con esfuerzo: KNN, QDA, PCR, PLS, la regresión polinómica con y sin interacciones y los splines. Quedan entonces los que se leen directamente, como la regresión lineal y la logística, Ridge, Lasso, LDA, Naive Bayes, el árbol de decisión y el SVM lineal. Esa exigencia cambia la elección en 103 de los 106 datasets, porque el mejor método disponible suele ser opaco, pero lo que se pierde por ella es poco. El mejor método interpretable queda de media a 0,027 del mejor si se descartan los métodos opacos, y a 0,061 si se descartan también los que solo se explican con esfuerzo. Lo que sí es grande es lo que pierde el recomendador con esas restricciones, 0,156 y 0,146, entre dos y seis veces más de lo inevitable. El motivo es que elige mal entre los métodos interpretables: aprendió a ordenar bien los métodos potentes, pero no los sencillos.

El estudio tiene varias limitaciones. La colección tiene pocos datasets en algunos regímenes, como los de más de 50 predictores, los de pocas observaciones por predictor o los que tienen valores faltantes de origen, así que las conclusiones son menos sólidas en esos casos. Los valores faltantes se eliminaron completamente al azar, que es el supuesto más neutro pero no refleja cómo suelen faltar los datos en la realidad, y en ese experimento la estrategia aprendida se entrena siempre con los datos completos, mientras que la fija combina los tres niveles. El random forest de Layer 2 lo elegí por razonamiento, sin compararlo con otros modelos como gradient boosting o una regresión lineal con interacciones, y alguno podría funcionar mejor. La ventaja de la estrategia aprendida en las tres primeras sugerencias se ha mantenido en todas las versiones del estudio sin llegar a ser significativa, y saber si es real exigiría más datasets. Por último, el benchmark no incluye GAM ni BART, que estaban previstos a través de R pero no llegaron a ejecutarse, ni un límite superior con el que medir cuánto margen de mejora queda. Para eso estaba previsto AMLBID, un sistema de recomendación de métodos publicado en 2022 (Garouani et al.), pero no se pudo reproducir porque falla con los datasets de la colección.

De ahí salen las líneas de trabajo futuro. La primera es probar otros modelos para Layer 2 y enseñarle a ordenar bien los métodos interpretables. También convendría ampliar la colección en los regímenes con pocos datasets, estudiar los valores faltantes con patrones más realistas y entrenar el recomendador con cada nivel de faltantes, e incorporar GAM, BART y un límite superior. Por último, cuando el usuario sube sus datos, se podrían calcular características directamente sobre ellos en lugar de usar solo las que permite el formulario. El Anexo 2 del cuaderno desarrolla estas ideas junto con las decisiones metodológicas del estudio.

Trazabilidad y código

El cuaderno [notebooks/tfm_graficos.ipynb](#) reproduce todas las cifras, los gráficos y los contrastes de este documento a partir de los resultados del benchmark guardados en el repositorio, y su Anexo 2 resume las decisiones metodológicas y el trabajo futuro. Las decisiones completas, con las alternativas que se descartaron y por qué, están en [DECISIONS.md](#). La evolución de los resultados a lo largo del estudio está en [FINDINGS.md](#), y la propuesta original del TFM en [propuesta-tfm.md](#). En la última fase del estudio se amplió el rango de valores de regularización que prueba Ridge al ajustarse, que antes era muy reducido, y se volvió a ejecutar

ese método. Las cifras de este documento son las posteriores a ese cambio, por lo que algunas difieren un poco de las que recoge la entrada F-003 de FINDINGS.md, aunque las conclusiones son las mismas. El código del benchmark y del recomendador está en `src/mlsandbox/`, `scripts/run_benchmark.py` y `scripts/package_model.py`.